Galaxy morphology classification with a convolutional neural network

Galaxy10 DECaLS: baselines, a custom CNN, transfer learning, and Grad-CAM

Philémon Vittet

July 2026

Abstract

We classify the morphology of 17 736 galaxies from the DESI Legacy Imaging Surveys (the Galaxy10 DECaLS dataset, with Galaxy Zoo labels) into 10 classes. The model is a convolutional network. To isolate what convolution brings, we compare three levels: baselines with no convolution (logistic regression, MLP), a custom CNN of 1 440 298 parameters, and a ResNet18 pretrained on ImageNet. The best model reaches an accuracy of 0.837 and a macro-F1 of 0.829 on the official test set. We interpret the predictions with Grad-CAM. We also measure the effect of augmentation and BatchNorm through ablations. One idea runs through the report: the translation equivariance of a CNN is a symmetry built into the architecture, in the spirit of symmetries in physics.

1 Introduction and context

The morphology of a galaxy (spiral, elliptical, barred, merging) carries physical information. It relates to the formation history, the environment, and the dynamics of the system. At survey scale, sorting millions of galaxies by hand is impossible. The Galaxy Zoo project asked hundreds of thousands of volunteers to label images. Our labels come from there. The question of this project is simple. Can a convolutional network reproduce this morphological sorting, automatically and reliably?

We answer it with a scale of three models of growing complexity. We go from a baseline that is blind to spatial structure up to a pretrained network. Then we interpret what the model learned. Historically, this family of models goes back to LeNet (LeCun, 1998), the first CNN trained by backpropagation for digit reading. It goes back to AlexNet (Krizhevsky, 2012), which showed at scale on ImageNet that deep CNNs win, and relaunched the whole field.

Related work. Automatic galaxy morphology classification with deep learning is now well established. Dieleman et al. (2015) won the Galaxy Zoo challenge with a CNN that used the rotation symmetries of the images explicitly. This confirmed the value of encoding the invariances of the problem into the architecture or the training. Galaxy10, derived from Galaxy Zoo and the DESI Legacy Imaging Surveys, is a standard benchmark for this task. Our contribution is not a new architecture. It is a controlled teaching study: isolate the value of convolution with a scale of models, then interpret the network and measure the effect of each regularization ingredient.

2 Data

Galaxy10 DECaLS holds 17 736 color images of size 256×256 (bands g, r, z from the DESI Legacy Imaging Surveys), split into 10 morphological classes. Table 1 gives the counts. The distribution is strongly imbalanced, by a factor close to 8 between the rarest class (Cigar Shaped, 334 images) and the most frequent (Round Smooth, 2 645). This imbalance is central to the

evaluation. Average per-channel color also differs slightly across classes, but the shape is the real signal, not the color.

id	class	n	id	class	n
0	Disturbed	1 081	5	Barred Spiral	2 043
1	Merging	1 853	6	Unbarred Tight Spiral	1 829
2	Round Smooth	2 645	7	Unbarred Loose Spiral	2 628
3	In-between Round Smooth	2 027	8	Edge-on without Bulge	1 423
4	Cigar Shaped Smooth	334	9	Edge-on with Bulge	1 873

Table 1: The 10 Galaxy10 DECaLS classes and their counts.

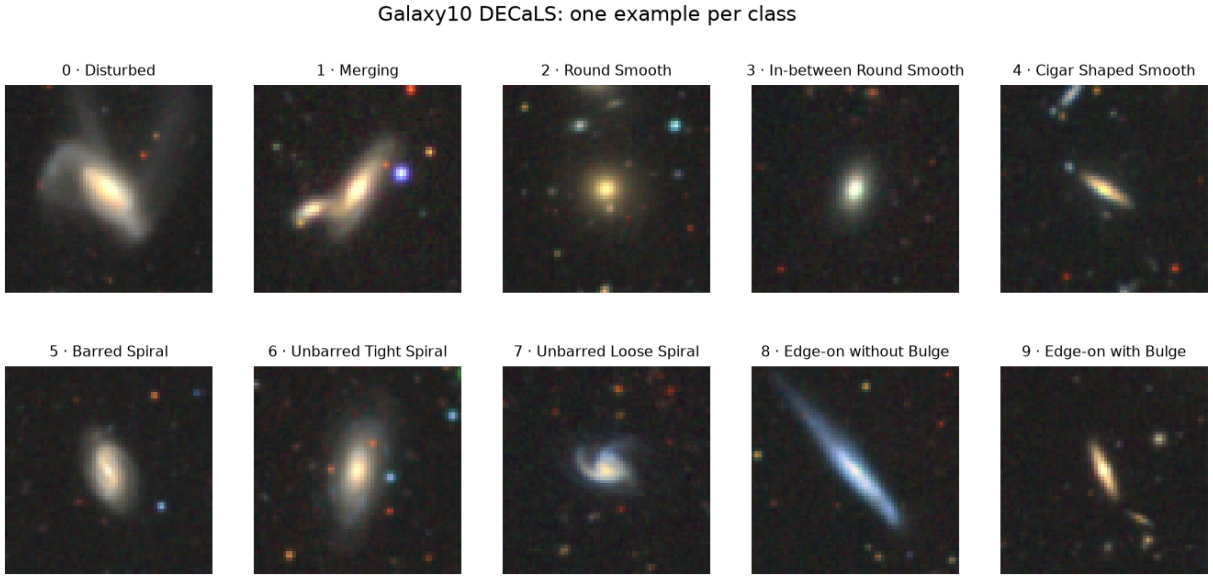


Figure 1: One example per class. Neighboring classes look alike: tight vs loose spirals, round vs cigar, edge-on with or without a bulge.

Split. The dataset ships with a `train` split (15 962 images) and a `test` split (1 774), which together make the 17 736 total. We keep the official test set as the final test set. We never use it during development. We take a stratified validation set, 15 % of the train split (about 2 395 images), with a fixed seed. All decisions (model choice, hyperparameters) are made on the validation set. This protocol avoids two classic leaks: deciding on the test set, and augmenting the data before the split.

3 Theory

3.1 From the perceptron to a deep network

A neural network stacks affine layers $z = Wx + b$ followed by a nonlinearity, usually the ReLU $\sigma(z) = \max(0, z)$. The last layer produces scores, turned into probabilities by the softmax $p_k = e^{z_k} / \sum_j e^{z_j}$. We train by minimizing the cross-entropy

$$\mathcal{L} = -\sum_k y_k \log p_k = -\log p_y. \tag{1}$$

This is the negative log-likelihood of the correct label under the model. Minimizing the cross-entropy is the same as maximizing the likelihood of the data. The gradients come from backprop-

agation, the chain rule applied in order, from the loss back to the weights. We update the weights with Adam.

Backpropagation computes $\partial \mathcal{L}/\partial W$ for each layer. It sends the gradient from the output back to the input. Each layer takes the incoming gradient, combines it with the local derivative of its operation, and passes the result to the previous layer. Adam keeps two running averages per parameter, one of the gradient and one of its square. It adapts the step per coordinate. This makes training robust to the initial learning rate. We also use a cosine schedule that shrinks the step over time, and early stopping when the validation macro-F1 stops improving. The learning rate is the most sensitive hyperparameter. Too large, the loss diverges. Too small, nothing learns.

3.2 The 2D convolution

A convolutional layer slides a small filter over the whole image. At each position it computes a local weighted sum:

$$(I * K)_{i,j} = \sum_m \sum_n I_{i+m,\, j+n}\, K_{m,n}. \tag{2}$$

This is the discrete convolution from signal processing, with the filter as the impulse response. Each output channel detects one local pattern (an edge, a blob, an orientation). Two properties follow.

Weight sharing. The same filter is used everywhere in the image. The number of parameters does not depend on the image size. It depends only on the filter size and the number of channels. This makes learning efficient and acts as a regularizer.

Translation equivariance and symmetries. The filter is applied the same way everywhere. So moving a pattern in the image moves its response by the same amount: $f(T_v\, x) = T_v\, f(x)$. The CNN encodes a symmetry of the problem in its architecture, instead of having to learn it. This is the same idea as in physics, where we impose the symmetries of a system (translation, rotation invariance) before solving the equations. A galaxy has no preferred position or orientation in the sky. So these invariances are physically justified. We complete the translation equivariance of the CNN with augmentation by rotations and flips (see the regularization section below).

3.3 Architecture and receptive field

We stack blocks of `Conv` $\rightarrow$ `BatchNorm` $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool`. Pooling lowers the spatial resolution and grows the receptive field. Deep layers see a larger part of the input image. This builds a hierarchy of features, from local edges to textures to global shapes (spiral arms, the bar, a merging companion). In numbers, each 3×3 convolution grows the receptive field by 2 pixels, and each MaxPool doubles it. After our four blocks, a unit of the last map sees a large part of the input. So it can respond to a whole morphological structure, not just an edge.

3.4 Sizing and parameter count

Our CNN takes a $3 \times 64 \times 64$ input. A 3×3 convolution with padding 1 keeps the spatial size. Each MaxPool halves it. Table 2 follows the sizes block by block.

3.5 Regularization

A network with more than a million parameters on a few thousand images overfits easily. We combine three regularizers. *Dropout* randomly turns off units during training. *BatchNorm* normalizes activations per mini-batch, which stabilizes and speeds up training. *Data augmentation* enriches the train set with label-preserving transforms: rotations 0–360°, horizontal and vertical flips, a light brightness jitter. As noted above, these transforms are physical invariances of the problem, not just noise.

layer	output (C, H, W)	conv weights (no bias)
input	$3 \times 64 \times 64$	–
block 1: Conv($3 \to 32$)+pool	$32 \times 32 \times 32$	$3 \cdot 32 \cdot 9 = 864$
block 2: Conv($32 \to 64$)+pool	$64 \times 16 \times 16$	$32 \cdot 64 \cdot 9 = 18\,432$
block 3: Conv($64 \to 128$)+pool	$128 \times 8 \times 8$	$64 \cdot 128 \cdot 9 = 73\,728$
block 4: Conv($128 \to 256$)+pool	$256 \times 4 \times 4$	$128 \cdot 256 \cdot 9 = 294\,912$
AdaptiveAvgPool(4×4)	$256 \times 4 \times 4$	–
FC $4096 \to 256$	256	$4096 \cdot 256 + 256 = 1\,048\,832$
FC $256 \to 10$	10	$256 \cdot 10 + 10 = 2\,570$

Table 2: Sizes and parameters of the custom CNN. Convolutions have no bias (BatchNorm provides the shift). Adding the 2 parameters per channel of BatchNorm ($64 + 128 + 256 + 512 = 960$), the total is 1 440 298 parameters. The first dense layer holds 73 % of them. The `AdaptiveAvgPool` fixes the head size, so this count is the same at 64 or 128 px.

3.6 Transfer learning

Instead of learning from scratch, we reuse a ResNet18 pretrained on ImageNet. ResNet adds residual connections $y = \mathcal{F}(x) + x$. They let the gradient flow and allow deep networks to train. We replace the last layer with 10 outputs. We proceed in two steps. First feature extraction: the backbone is frozen and we train only the head. Then fine-tuning: everything is unfrozen with a smaller learning rate.

4 Experimental setup

Images are resized to 64 px (baselines and CNN) and 128 px (ResNet), then normalized channel by channel with statistics computed on the train set only. Augmentation is applied to the train set only, never to validation or test. We handle the imbalance by weighting the cross-entropy by the inverse class frequency. The evaluation favors macro-F1, which treats all classes equally, over accuracy alone. Training uses Adam, a cosine schedule, and early stopping on the validation macro-F1. The code is a reusable Python package plus a self-contained notebook. Runs work in local (Mac, MPS backend) and on Colab (T4 GPU), with automatic device detection.

Protocol and reproducibility. The validation draw, the weight init, and the batch shuffling are controlled by a fixed seed (42). The validation set is 15 % of the train split (about 2 395 images). The official test set has 1 774 images. Batches hold 128 images. The optimizer is Adam with a cosine schedule. Early stopping has a patience of 15 epochs. The normalization statistics, the best architecture, and the hyperparameters are frozen from the validation set, then applied once to the test set. The whole pipeline is deterministic for a given seed, so the results are reproducible.

5 Implementation notes

The core of the code is short. A convolution block chains a bias-free convolution, a BatchNorm, a ReLU, and a pooling. The CNN stacks four such blocks and a dense head.

```
def _conv_block(in_c, out_c, use_bn=True):
    layers = [nn.Conv2d(in_c, out_c, 3, padding=1, bias=not use_bn)]
    if use_bn:
        layers.append(nn.BatchNorm2d(out_c))
    layers += [nn.ReLU(inplace=True), nn.MaxPool2d(2)]
    return nn.Sequential(*layers)
```

Grad-CAM reduces to two hooks (to capture the activations and gradients of the last convolution) and a weighted sum followed by a ReLU.

```
out = model(x) # x: [1, 3, H, W]
model.zero_grad()
out[0, class_idx].backward()
weights = gradients.mean(dim=(2, 3), keepdim=True) # per-channel importance
cam = F.relu((weights * activations).sum(1, keepdim=True))
cam = F.interpolate(cam, size=x.shape[-2:], mode="bilinear")
```

6 Experiments and results

6.1 The baseline → CNN → ResNet scale

Table 3 and Figure 2 sum up what convolution brings. The baselines are blind to spatial structure and stay weak. The custom CNN improves a lot. Transfer learning adds one more step.

model	accuracy	macro-F1	weighted F1
Logistic regression (pixels)	0.304	0.273	0.295
MLP, 2 layers (pixels)	0.508	0.474	0.504
Custom CNN	0.782	0.766	0.778
ResNet18 (fine-tuned)	0.837	0.829	0.837

Table 3: Performance on the official test set, never used during development. Per-class precision and recall are in the notebook.

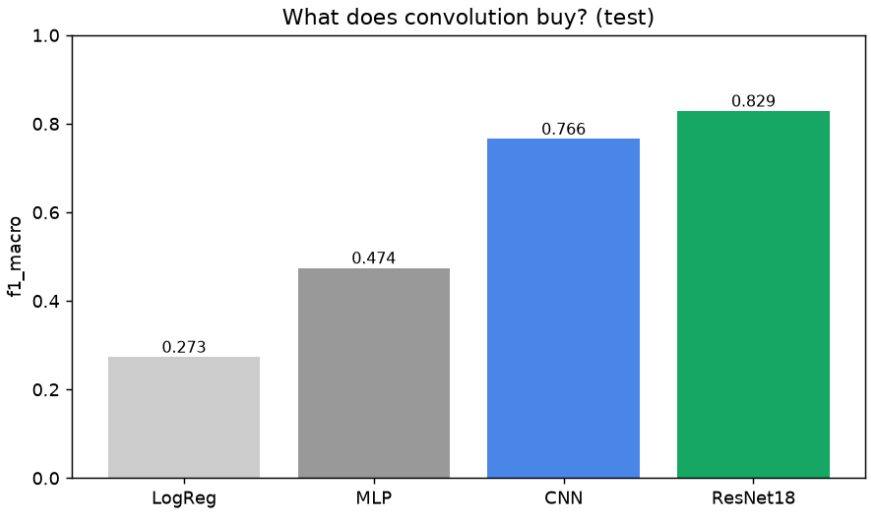


Figure 2: Test macro-F1, from baselines to ResNet. Convolution, then pretraining, explain most of the gain.

6.2 Tuning and learning curves

We swept the learning rate on the validation set. On a short budget (15 epochs), 10^{-3} gave the best macro-F1. The reported CNN was trained for 80 epochs at $\mathrm{lr} = 10^{-3}$. The sweep is exploratory: its 15-epoch scores are not directly comparable to the final 80-epoch model. It mainly shows that performance is stable between 3×10^{-4} and 10^{-3}, and drops at 3×10^{-3}. Figure 3 shows the loss and the validation macro-F1 over epochs, with the epoch kept by early stopping.

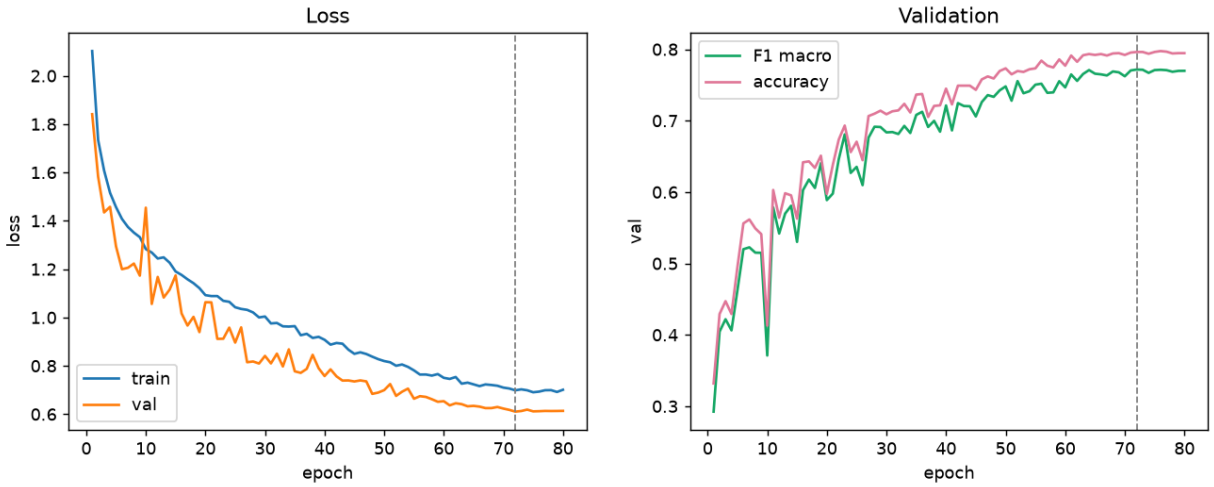


Figure 3: Learning curves of the custom CNN (train/val loss and validation metrics). The vertical line marks the epoch with the best macro-F1.

6.3 Per-class analysis and confusions

The confusion matrix (Figure 4) and the per-class F1 (Figure 5) show where the model fails. Smooth and edge-on classes are the best recognized. Edge-on with Bulge gets the top F1, thanks to its clean and distinctive silhouette. The hardest class is Disturbed. Disturbed galaxies overlap with mergers and form a mixed category. This is more an ambiguity of definition than a lack of examples. Tight and loose spirals also mix with each other. The rare Cigar Shaped class stays fragile. So the confusions concentrate, as expected, between morphologically close classes.

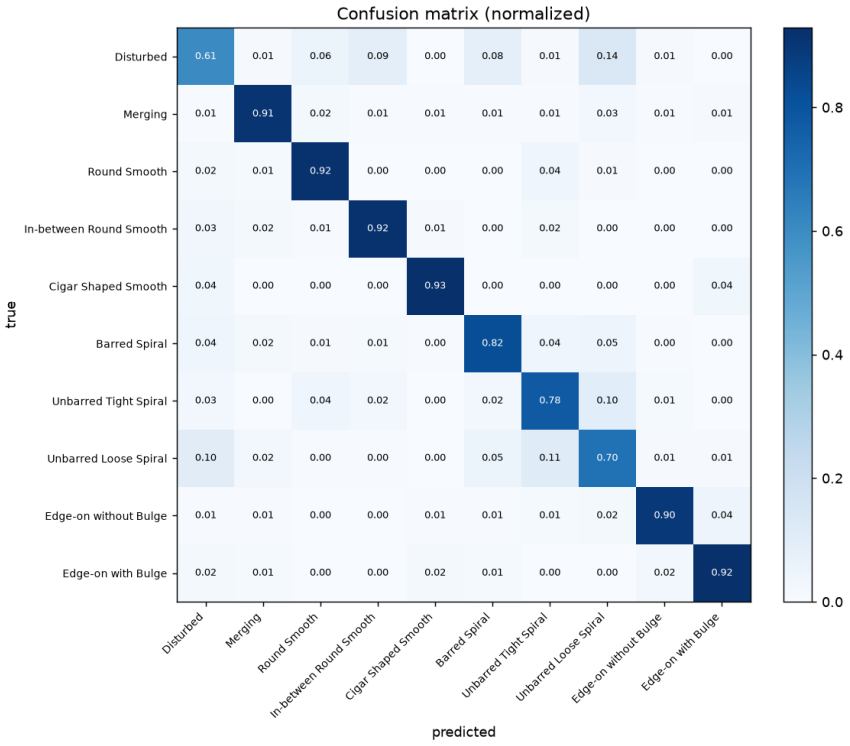


Figure 4: Confusion matrix, normalized per row (ResNet18, test).

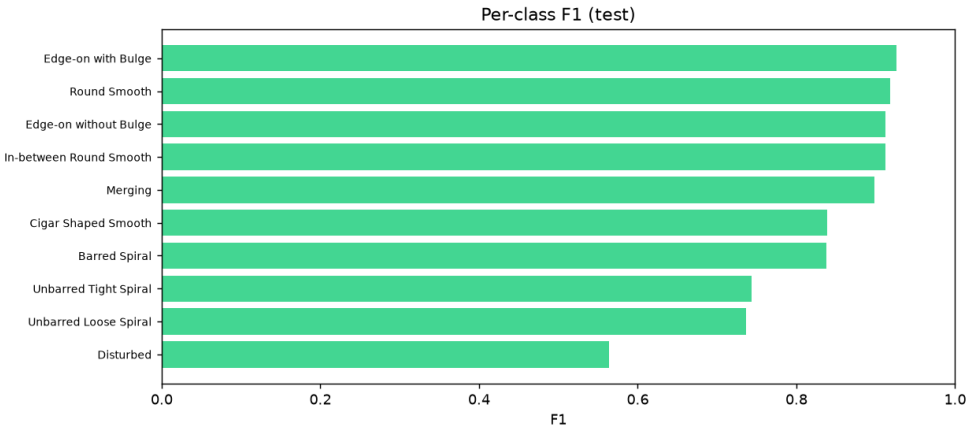


Figure 5: Per-class F1 on the test set (ResNet18). Rare and ambiguous classes are at the bottom.

class	F1	class	F1
Edge-on with Bulge	0.927	Cigar Shaped Smooth	0.839
Round Smooth	0.918	Barred Spiral	0.838
Edge-on without Bulge	0.913	Unbarred Tight Spiral	0.743
In-between Round Smooth	0.912	Unbarred Loose Spiral	0.736
Merging	0.898	Disturbed	0.564

Table 4: Per-class F1 (ResNet18, test), best to worst. The smooth and edge-on classes lead; the disturbed and loose-spiral classes trail.

6.4 Interpretation with Grad-CAM

Grad-CAM weights the feature maps of the last convolution by the mean gradient of the class score, applies a ReLU, then rescales to the image size. We get a heatmap of the regions that drive the prediction. Formally, for class c and feature maps A^k of the last convolution, we compute the weights $\alpha_k^c = \frac{1}{Z}\sum_{i,j} \partial y^c / \partial A_{i,j}^k$ (spatial mean of the gradient), then the map $L^c = \mathrm{ReLU}\big(\sum_k \alpha_k^c A^k\big)$, rescaled to the image size. Figure 6 shows correct cases and errors. On correct predictions, the network focuses on the relevant structures (spiral arms, the bar, a merging companion). On errors, the map sometimes rests on a neighbor or an artifact.

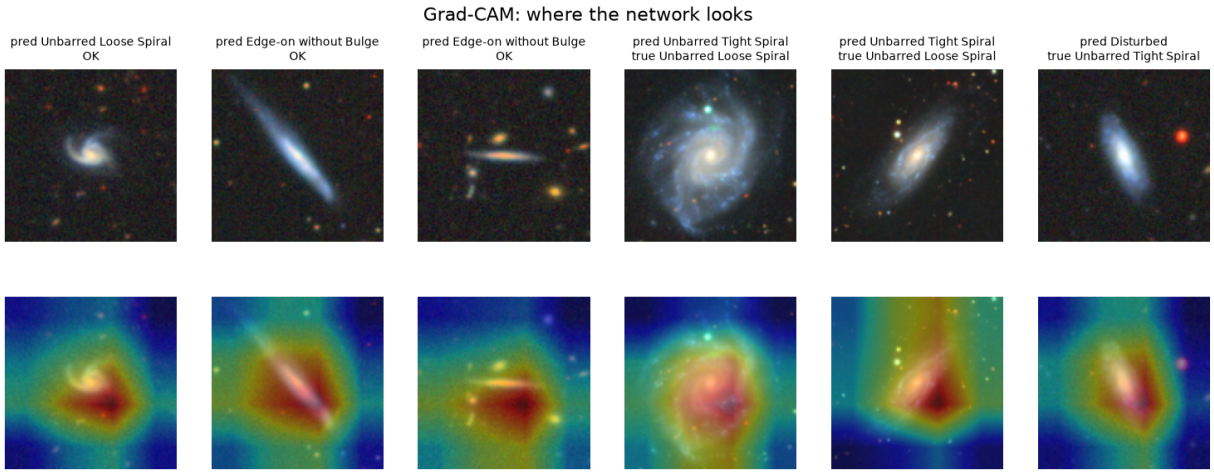


Figure 6: Grad-CAM on the ResNet18 (top row: image; bottom row: overlaid map). Correct predictions, then errors.

6.5 Ablations

We remove one ingredient at a time to measure its effect (test macro-F1). The full CNN reaches 0.766. The variant without augmentation reaches 0.708. The variant without BatchNorm reaches 0.787. Augmentation clearly helps. The full model beats the no-augmentation variant by about six F1 points, in line with rotations and flips that match the physical invariances of galaxies. Removing BatchNorm is the surprise. It raises the test F1 a little. Two readings fit. A single run can hide a small gap in the noise, and a clean ablation should average over several seeds. BatchNorm still keeps its main value, faster and more stable convergence, which the learning curves confirm. Its benefit here is about optimization, not the final test F1.

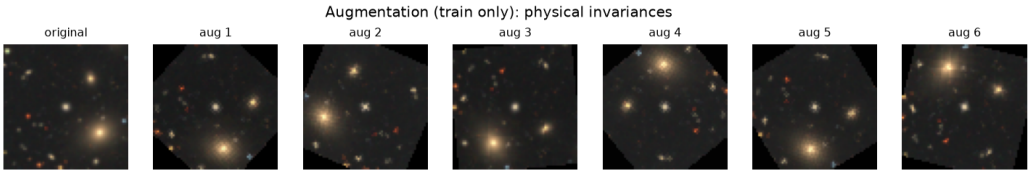


Figure 7: Effect of augmentation on one galaxy (rotations, flips, jitter). The label is kept, the look changes.

7 Limits

Three main limits. The Galaxy Zoo labels are noisy. They aggregate volunteer votes, and the borders between neighboring classes are fuzzy, which caps the reachable performance. The imbalance penalizes rare classes despite the weighted loss. And the training cost limits the search. We worked at 64 and 128 px rather than 256, and the hyperparameter space was only partly swept.

Several directions would extend this work. We could train at 256 px to use the native resolution. We could replace the loss weighting by oversampling of rare classes or a focal loss. We could estimate prediction uncertainty (for example by test-time dropout) to flag ambiguous cases instead of forcing a decision. Comparing several pretrained backbones, and using rotation invariance in the architecture explicitly (equivariant layers), are two more directions in the spirit of this project.

8 Conclusion

The baseline $\rightarrow$ CNN $\rightarrow$ ResNet scale answers the starting question. Convolution, then transfer learning, bring most of the performance, and the best model reproduces the Galaxy Zoo morphological sorting well. Grad-CAM confirms that the network relies on the expected morphological structures. The ablations measure the effect of each ingredient. The idea to keep is that a CNN works on images largely because of a symmetry built in by design, translation equivariance, extended by an augmentation that encodes the physical invariances of the problem.